SQL Tutorial (Detailed Version)

Chapter 1: SQL HOME

SQL (Structured Query Language) is the industry-standard language used to interact with relational databases. It is supported by almost all relational database management systems (RDBMS) like MySQL, SQL Server, Oracle, and PostgreSQL.

Key Features:

- SQL allows creation of databases, tables, views, and indexes.
- SQL helps in data manipulation (Insert, Update, Delete, Select).
- SQL ensures data security and integrity with constraints.
- SQL supports transactions and rollback mechanisms.

Chapter 2: SQL Intro

- SQL is declarative: you specify **what** you want, not **how** to get it.
- Developed in the 1970s by IBM and standardized by ANSI.
- It is widely used in **business intelligence, web applications, analytics, and ETL processes**.

Benefits:

- Platform independent.
- Easy to learn with English-like commands.
- Can handle large amounts of data efficiently.

Chapter 3: SQL Syntax

A typical SQL statement follows the structure:

- SELECT column1, column2
- FROM table_name
- WHERE condition
- ORDER BY column1;

Rules:

1. SQL keywords (SELECT, FROM, WHERE) are case-insensitive.
2. Text values must be enclosed in single quotes `'value'`.
3. Numeric values are written without quotes.
4. A semicolon `;` is used to end SQL statements (mandatory in some systems).

Chapter 4: SQL SELECT Statements

4.1 SELECT

Fetches columns from a table.

- SELECT CustomerName, City FROM Customers;

4.2 SELECT DISTINCT

Eliminates duplicates.

- SELECT DISTINCT Country FROM Customers;

4.3 WHERE Clause

Filters records.

- SELECT * FROM Customers WHERE Country='Mexico';

4.4 ORDER BY

Sorts results in ascending (default) or descending order.

- SELECT * FROM Products ORDER BY Price DESC;

4.5 AND / OR / NOT Operators

- -- Customers in Spain and Madrid
- SELECT * FROM Customers WHERE Country='Spain' AND City='Madrid';
- -- Customers from Germany or Spain
- SELECT * FROM Customers WHERE Country='Germany' OR Country='Spain';
- -- Customers not from Spain
- SELECT * FROM Customers WHERE NOT Country='Spain';

Chapter 5: SQL Data Manipulation

5.1 INSERT INTO

Adds new rows.

- INSERT INTO Customers (CustomerName, City, Country)
- VALUES ('Cardinal', 'Stavanger', 'Norway');

5.2 NULL Values

- NULL = No value, not zero or empty.

- Use `IS NULL` or `IS NOT NULL`.
- SELECT * FROM Customers WHERE Address IS NULL;

5.3 UPDATE

Modifies rows.

- UPDATE Customers SET City='Frankfurt' WHERE CustomerID=1;

5.4 DELETE

Deletes rows.

- DELETE FROM Customers WHERE CustomerID=1;

⚠️ Warning: Without WHERE clause, **all rows** are affected.

Chapter 6: SQL Advanced Queries

6.1 SELECT TOP / LIMIT

Restricts number of records returned.

- SELECT TOP 5 * FROM Customers; -- SQL Server
- SELECT * FROM Customers LIMIT 5; -- MySQL

6.2 Aggregate Functions

Perform calculations on sets of data:

- `MIN()` – Smallest value
- `MAX()` – Largest value
- `COUNT()` – Number of rows
- `SUM()` – Total sum
- `AVG()` – Average value

Example:

- SELECT MIN(Price) AS MinPrice, MAX(Price) AS MaxPrice, AVG(Price) AS AvgPrice FROM Products;

6.3 LIKE & Wildcards

- `%` = zero or more characters
- `_` = one character
- SELECT * FROM Customers WHERE CustomerName LIKE 'A%';

6.4 IN

- SELECT * FROM Customers WHERE Country IN ('Germany','France','UK');

6.5 BETWEEN

- SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;

6.6 Aliases

Rename tables or columns.

- SELECT CustomerID AS ID, CustomerName AS Name FROM Customers;

Chapter 7: SQL Joins

7.1 INNER JOIN

Returns records with matching keys.

- SELECT Orders.OrderID, Customers.CustomerName
- FROM Orders
- INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;

7.2 LEFT JOIN

Returns all from left table and matches from right.

- SELECT Customers.CustomerName, Orders.OrderID
- FROM Customers
- LEFT JOIN Orders ON Customers.CustomerID=Orders.CustomerID;

7.3 RIGHT JOIN

Returns all from right table and matches from left.

- SELECT Orders.OrderID, Employees.LastName
- FROM Orders
- RIGHT JOIN Employees ON Orders.EmployeeID=Employees.EmployeeID;

7.4 FULL JOIN

Returns rows if a match exists in either table.

- SELECT Customers.CustomerName, Orders.OrderID
- FROM Customers

- FULL OUTER JOIN Orders ON Customers.CustomerID=Orders.CustomerID;

7.5 SELF JOIN

Join table with itself.

- SELECT A.CustomerName, B.CustomerName, A.City
- FROM Customers A, Customers B
- WHERE A.City=B.City AND A.CustomerID<>B.CustomerID;

Chapter 8: SQL Set Operations

UNION

Removes duplicates.

- SELECT City FROM Customers
- UNION
- SELECT City FROM Suppliers;

UNION ALL

Keeps duplicates.

- SELECT City FROM Customers
- UNION ALL
- SELECT City FROM Suppliers;

Chapter 9: SQL Grouping & Filtering

GROUP BY

- SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country;

HAVING

Filters aggregated results.

- SELECT Country, COUNT(CustomerID) FROM Customers GROUP BY Country HAVING COUNT(CustomerID)>5;

EXISTS

Checks if subquery returns rows.

- SELECT * FROM Customers WHERE EXISTS (SELECT * FROM Orders WHERE Customers.CustomerID=Orders.CustomerID);

ANY & ALL

- SELECT ProductName FROM Products WHERE Price > ANY (SELECT Price FROM Products WHERE CategoryID=1);

Chapter 10: SQL Table Operations

SELECT INTO

Copy data into a new table.

- SELECT * INTO CustomersBackup FROM Customers;

INSERT INTO SELECT

Copy rows from one table into another.

- INSERT INTO CustomersBackup SELECT * FROM Customers;

CASE Expression

Conditional logic.

- SELECT OrderID, Quantity,
- CASE
- WHEN Quantity>30 THEN 'Large'
- WHEN Quantity BETWEEN 10 AND 30 THEN 'Medium'
- ELSE 'Small'
- END AS OrderSize
- FROM OrderDetails;

Null Functions

Replace NULLs.

- SELECT ISNULL(Address, 'No Address') FROM Customers; -- SQL Server

Chapter 11: SQL Programmability

Stored Procedures

Reusable queries.

- CREATE PROCEDURE SelectAllCustomers AS SELECT * FROM Customers;
- EXEC SelectAllCustomers;

Comments

- -- Single line comment
- /* Multi-line comment */

Operators

- Arithmetic: +, -, *, /, %
- Comparison: =, <, >, <=, >=, <>
- Logical: AND, OR, NOT

Chapter 12: SQL Database Management

Create Database

- CREATE DATABASE TestDB;

Drop Database

- DROP DATABASE TestDB;

Backup Database

- BACKUP DATABASE TestDB TO DISK='D:\backup\testdb.bak';

Chapter 13: SQL Tables

Create Table

- CREATE TABLE Customers (
- CustomerID int PRIMARY KEY,
- CustomerName varchar(255) NOT NULL,
- Country varchar(50)
-);

Drop Table

- DROP TABLE Customers;

Alter Table

- ALTER TABLE Customers ADD Email varchar(255);
- ALTER TABLE Customers DROP COLUMN Email;

Chapter 14: SQL Constraints

Constraints ensure data integrity.

NOT NULL

- CREATE TABLE Persons (ID int NOT NULL, Name varchar(255) NOT NULL);

UNIQUE

- CREATE TABLE Persons (ID int UNIQUE, Email varchar(255) UNIQUE);

PRIMARY KEY

- CREATE TABLE Persons (ID int PRIMARY KEY, Name varchar(255));

FOREIGN KEY

- CREATE TABLE Orders (
- OrderID int PRIMARY KEY,
- CustomerID int,
- FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
-);

CHECK

- CREATE TABLE Products (ID int PRIMARY KEY, Price int CHECK (Price>0));

DEFAULT

- CREATE TABLE Orders (ID int, OrderDate date DEFAULT GETDATE());

INDEX

- CREATE INDEX idx_customername ON Customers(CustomerName);

AUTO INCREMENT

- CREATE TABLE Persons (
- ID int AUTO_INCREMENT PRIMARY KEY, -- MySQL
- Name varchar(255)
-);

Chapter 15: SQL Advanced Topics

Dates

- SELECT * FROM Orders WHERE OrderDate BETWEEN '2024-01-01' AND '2024-12-31';

Views

- CREATE VIEW GermanyCustomers AS SELECT * FROM Customers WHERE Country='Germany';

SQL Injection

Malicious input:

- SELECT * FROM Users WHERE Username='' OR '1'='1';

Prevention: Always use **prepared statements**.

SQL Hosting

- Local Servers
- Cloud services (AWS RDS, Azure SQL, GCP Cloud SQL)

SQL Data Types

- **Numeric**: INT, DECIMAL, FLOAT
- **String**: CHAR, VARCHAR, TEXT
- **Date/Time**: DATE, DATETIME, TIMESTAMP
- **Boolean**: BIT, BOOLEAN